



Project Details:

Project Title - CrowdCount-People Counting Using Video Analytics

Project Purpose -

- Detect, track, and count people in real time from live or recorded video feeds.
- Help organizations and event managers monitor crowd density and ensure safety compliance.
- Provide smart alerts, visual dashboards, and live statistics for better crowd management and decision-making.

Tools and Technologies - Python, Flask, OpenCV, YOLOv8, SQLite, JavaScript, Chart.js

Mentor - Ms.Umme Asma S

Batch Number – 3

Start Date - 22-September-2025

Intern Name - U Surya Sasikanth

GitHub Repository -

https://github.com/sasi-upparapalli/infosys_crowdcount_october2025_sasi-upparapalli

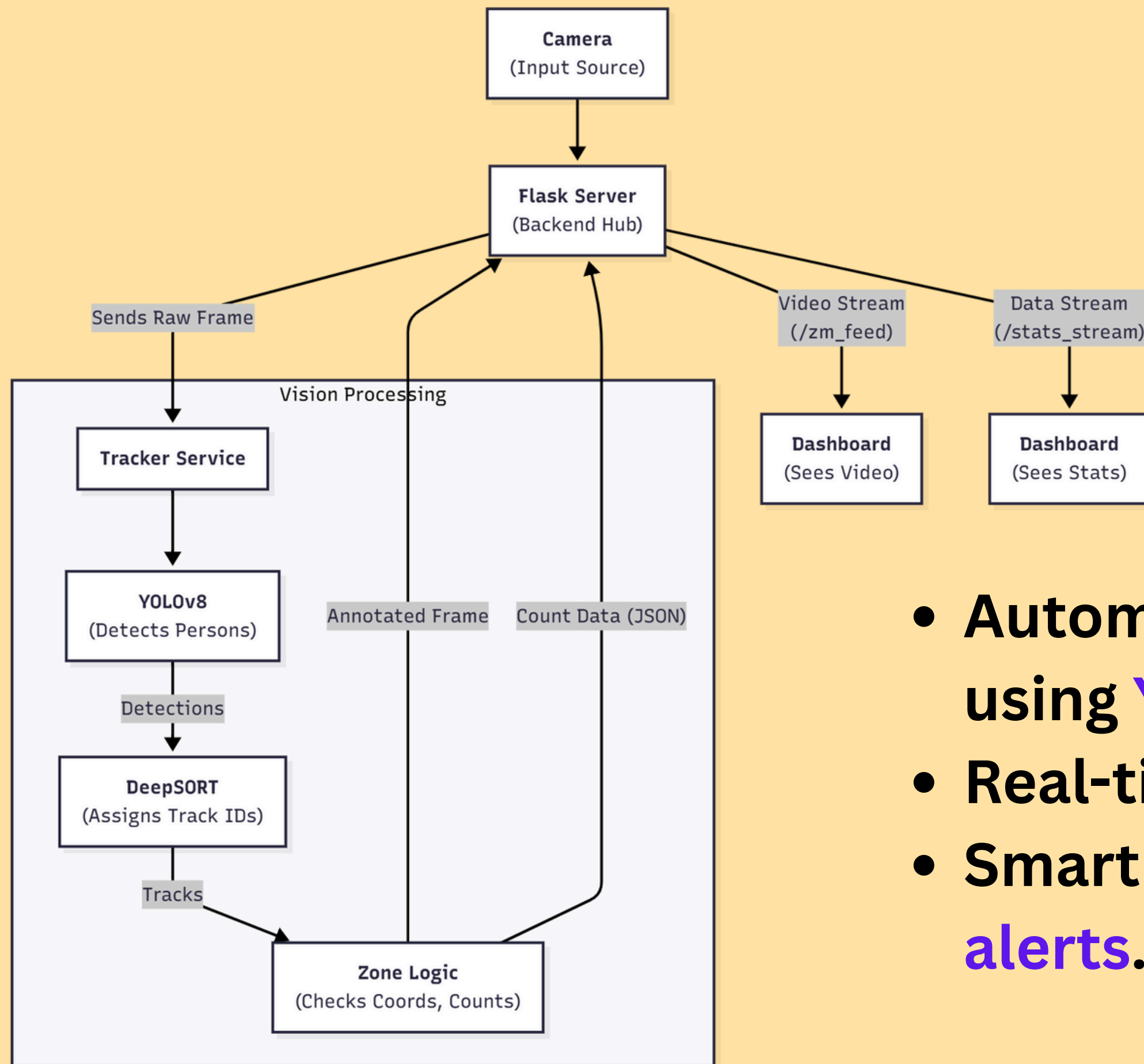
Project Overview-

<https://sasi-upparapalli.github.io/Crowd-Count/>

Problem Statement

- ⚠️ Manual crowd monitoring is **inefficient and labor-intensive**.
- 🕒 Lack of real-time data leads to **delayed safety** actions.
- 👁️ **No accurate way** to track occupancy dynamically in public or closed spaces.
- 💰 **High cost and human error** in traditional crowd management systems.
- 🔑 **Limited integration** with smart surveillance networks reduces scalability.





Proposed Solution

- Automated detection and counting using **YOLOv8**.
- Real-time tracking with **Deep SORT**.
- Smart dashboard for live **stats and alerts**.

Real-World Implementation Areas

Area of Implementation	Use Case / Purpose
 Educational Institutions	Monitor crowd density in campuses or exam halls.
 Shopping Malls & Public Spaces	Manage foot traffic and ensure safety compliance.
 Corporate Buildings	Track occupancy in lobbies, corridors, and meeting areas.
 Transport Hubs	Monitor passenger flow in stations, airports, and terminals.
 Events & Stadiums	Maintain safe crowd limits during large gatherings.
 Smart City Surveillance	Integrate with CCTV for real-time crowd analytics.



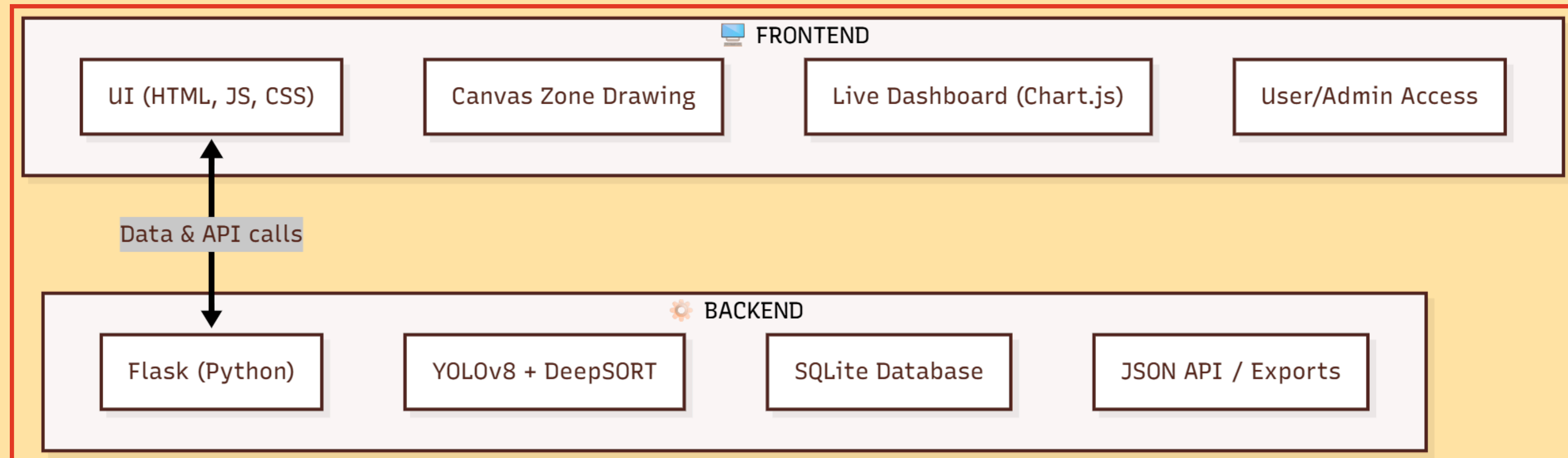
Project Architecture

Frontend

- Built using **HTML, CSS, and JavaScript** for an interactive user interface.
- **Canvas API** used for drawing and managing detection zones.
- **Chart.js** visualizes real-time crowd data and alerts.
- Provides user dashboards and admin panels for control and monitoring.

Backend

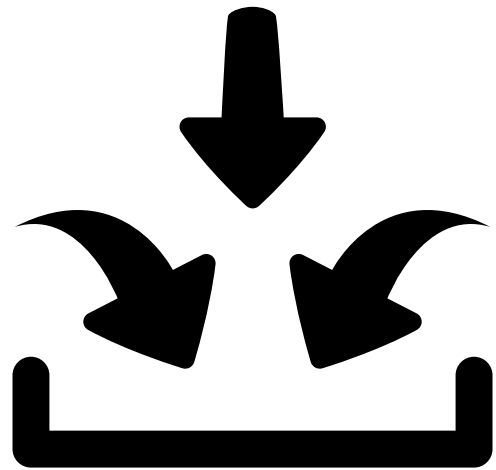
- Developed using **Flask (Python)** as the main web server.
- Handles video stream processing and model inference (**YOLOv8 + Deep SORT**).
- Communicates with SQLite3 database to save zones, logs, and user data.
- Provides **REST APIs** for uploading videos, tracking, exporting reports, etc.



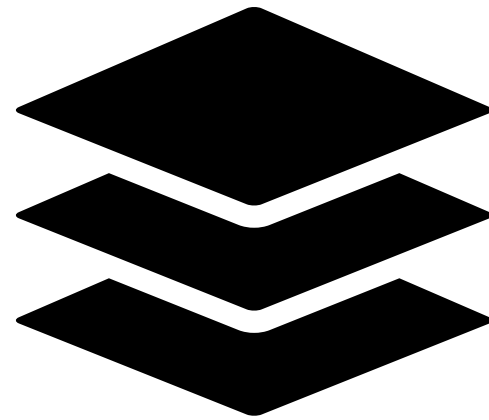
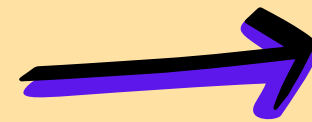
Tech Stack

Category	Tools & Technologies Used
Programming Language	Python, JavaScript
Backend Framework	Flask
Frontend Technologies	HTML5, CSS3, Chart.js, Canvas API
Machine Learning / AI Model	YOLOv8 (Object Detection), Deep SORT (Tracking)
Database	SQLite3
Visualization & UI	Chart.js, Flask Templates (Jinja2)
Libraries & Packages	OpenCV, Ultralytics, FPDF, JWT, Werkzeug
Version Control & Deployment	Git, Render / Local Server
Environment	Python Virtual Environment (venv)

System Workflow



 **Input Layer** – Captures live or uploaded video feeds as the main data source.



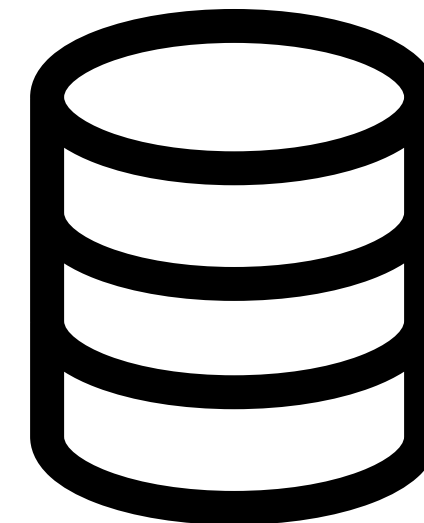
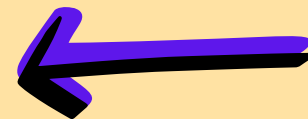
 **Processing Layer** – Flask routes frames to YOLOv8 for detection; Deep SORT tracks individuals with unique IDs.



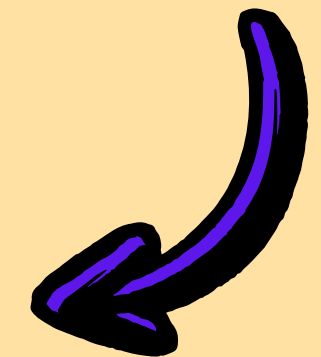
 **Analytics Layer** – Counts people per zone, updates in real time, and triggers alerts on threshold breach.



 **Presentation Layer** – Live dashboard displays analytics, charts, and admin controls.



 **Storage Layer** – Saves users, zones, logs, and reports in an SQLite database.



Key Features of CrowdCount

```
graph TD; A((Key Features of CrowdCount)) --- B[Role-Based Access Control - Secure login with admin and user privileges]; A --- C[Threshold Alerts - Notifies users when a zone exceeds its defined capacity.]; A --- D[Interactive Dashboard - Visualizes data using Chart.js (line & bubble charts).]; A --- E[Data Export - Download analytics and logs in CSV or PDF format.]; A --- F[Multi-Camera Integration - Add or manage multiple IP or webcam feeds.]; A --- G[Real-Time Updates - Automatic refresh of zone counts on the dashboard within seconds.]; A --- H[Custom Zone Creation - Draw and manage multiple regions of interest on video feeds.]; A --- I[Live People Counting - Real-time detection and tracking using YOLOv8.];
```

Role-Based Access Control – Secure login with admin and user privileges

Threshold Alerts – Notifies users when a zone exceeds its defined capacity.

Interactive Dashboard – Visualizes data using Chart.js (line & bubble charts).

Data Export – Download analytics and logs in CSV or PDF format.

Multi-Camera Integration – Add or manage multiple IP or webcam feeds.

Real-Time Updates – Automatic refresh of zone counts on the dashboard within seconds.

Custom Zone Creation – Draw and manage multiple regions of interest on video feeds.

Live People Counting – Real-time detection and tracking using YOLOv8.

Execution Steps

Setup Environment

- Install dependencies using **pip install -r requirements.txt** and ensure Python 3.9+ with webcam or video input.

Initialize Database

- Run **python app.py** to auto-create video_zone.db and configure admin credentials.

Load / Stream Video

- Use **webcam or upload video** from dashboard to start live feed.

Detection & Tracking

- **YOLOv8** detects people; **Deep SORT** tracks unique IDs in real time.

Live Counting & Alerts

- Zones update counts automatically and trigger alerts on threshold breaches.

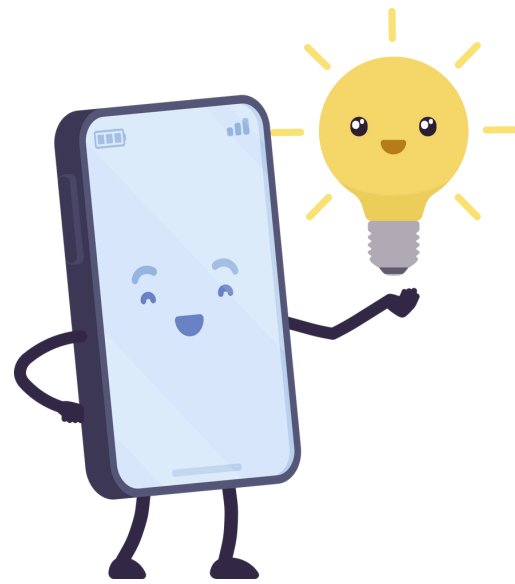
View Dashboard & Export

- Monitor analytics, logs, and export data (CSV/PDF) from the dashboard.

Mathematics & Formulas Behind the Project

Concept / Component	Mathematical Formula / Logic	Purpose in Project
Object Detection (YOLOv8)	$P(\text{Class}) \times \text{IoU}$ — Combines class probability and Intersection over Union to detect people accurately.(tracker_service.py)	Identifies people in each video frame using bounding boxes.
IoU (Intersection over Union)	$\text{IoU} = (\text{Area of Overlap}) / (\text{Area of Union})$ (tracker_service.py)	Measures how well the detected bounding box matches the actual object.
Centroid Tracking	$C(x, y) = ((x_1 + x_2)/2, (y_1 + y_2)/2)$ (tracker_service.py)	Finds the center point of each detected person to track movement.
Distance Measurement	$D = \sqrt{((x_2 - x_1)^2 + (y_2 - y_1)^2)}$ (tracker_service.py)	Calculates the movement distance between frames for unique ID tracking.
People Counting Logic	$\text{Count} = \Sigma (\text{Unique IDs in Zone})$ (tracker_service.py)+(script.js)	Counts unique individuals entering or present in a defined zone.
Zone Density	$\text{Density} = (\text{People Count}) / (\text{Zone Area})$ (app.py)	Helps estimate crowd density for safety and alert generation.
Alert Threshold	If $\text{Count} \geq \text{Threshold} \rightarrow \text{Trigger Alert}$ (app.py)+(Live.html)	Sends warning when people count exceeds predefined safe limit.

Future Enhancements



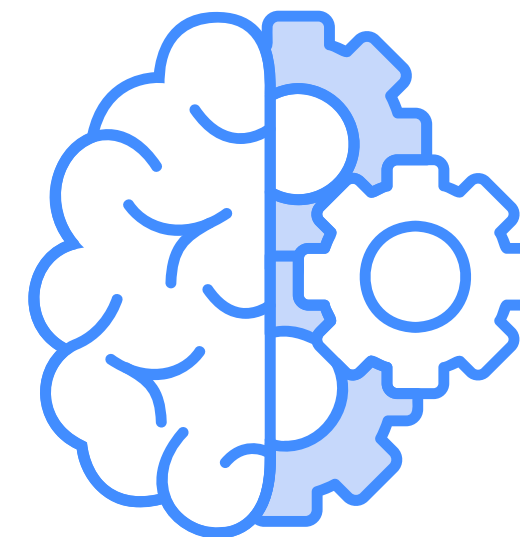
Develop a **mobile friendly** dashboard accessible via smartphones for real-time monitoring and alerts on the go.



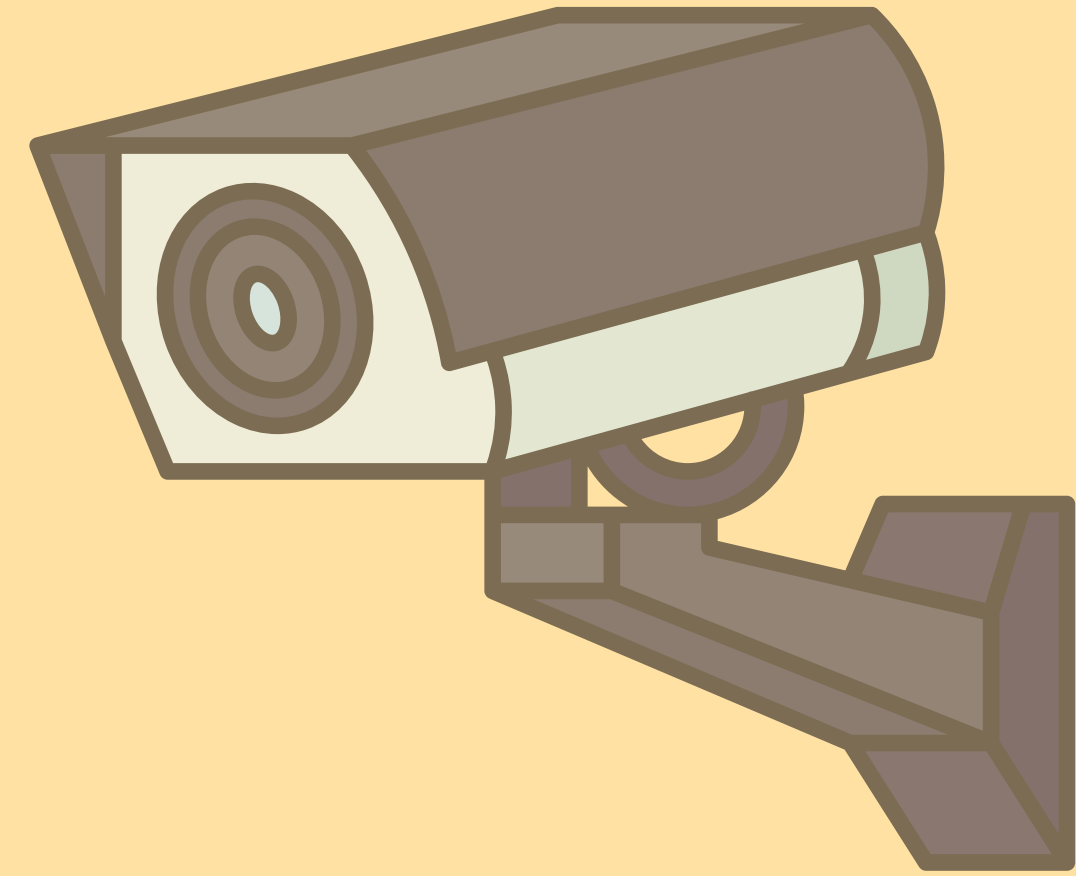
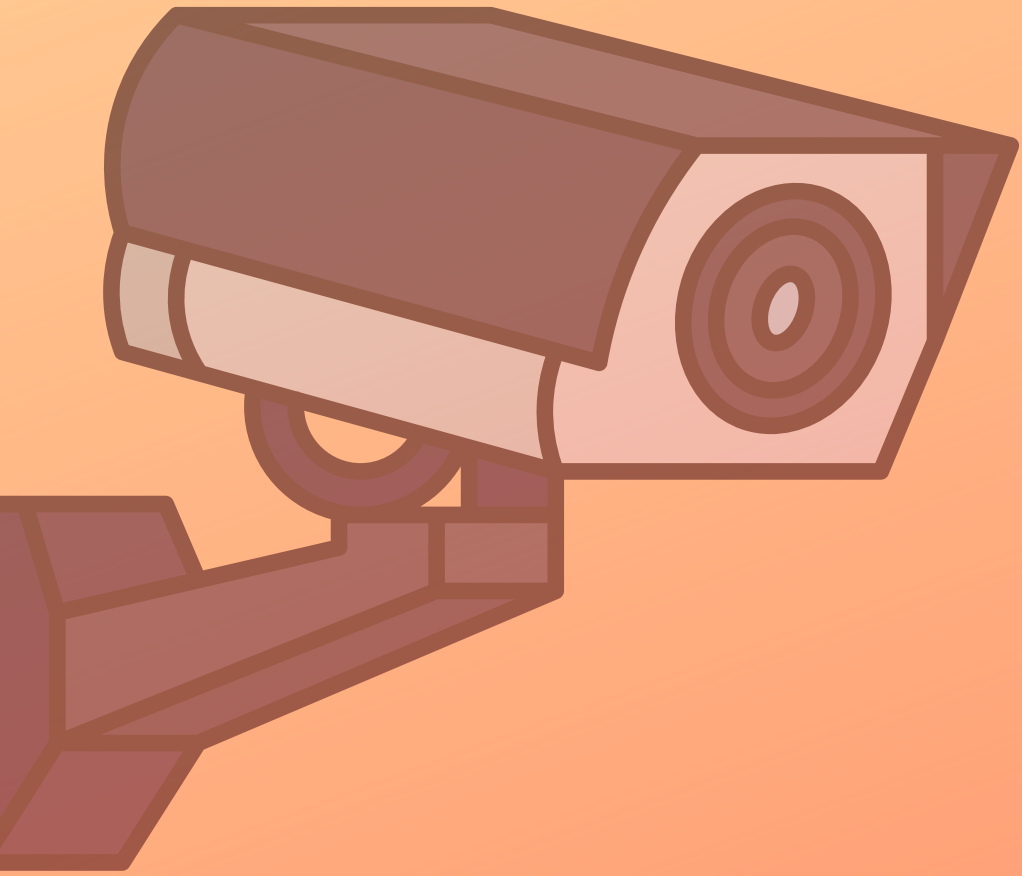
Add **multilingual options, voice assistance, and accessibility** features to make the system inclusive and usable by diverse users.



Add automated **voice or sound alerts** when crowd thresholds are exceeded, improving real-time responsiveness.



Implement **deep learning models** to automatically identify unusual crowd movements, panic situations, or suspicious activities in real time.



Thank
you

